Autonomous Cave Exploration

Autonomous Systems Project Report

Group:12
Samuel Ribaudo (03821248)
Hong Yan Jun (03813507)
Fuzail Hamid Banday (03811887)
Firdavs Shukurilloev (03796553)
Om Anant Kulkarni (03812156)

Technical University of Munich

March 4, 2026

Abstract

This report presents the design and implementation of an autonomous cave exploration and lantern detection. The system integrates a high level state machine, mapping, planning, control and perception to safely explore unknown subterranean spaces while also detecting lanterns and logging their locations.

Contents

1 Introduction **3**

2 System Architecture **3**
2.1 Implemented ROS Packages . . . 3

3 Trajectory Generation (trajectory_generator_pkg) **3**
3.1 Method . . . 4
3.2 Publishing . . . 4

4 Perception and Mapping (perception) **4**
4.1 Depth to Point Cloud . . . 4
4.2 Voxel Grid Mapping . . . 4

5 Lantern Detection and Hunting **5**
5.1 Detection . . . 5
5.2 Hunting Strategy . . . 5

6 Exploration Strategy (exploration_pkg) **6**
6.1 Node Architecture . . . 6
6.1.1 Frontier Detection . . . 6
6.1.2 Spatial Clustering . . . 6
6.2 Advanced Exploration Strategy . . . 6
6.2.1 Dynamic Threshold Reduction . . . 6
6.2.2 Heading Bias and Scoring . . . 6
6.3 Navigation and Safety . . . 7
6.3.1 Forbidden Radius Guard . . . 7
6.3.2 Passive Logging Mode . . . 7
6.4 Graph Logic and Backtracking . . . 7
6.5 Communication Interface . . . 7

7 Mission State Machine (decision_making) **8**
7.1 State Definition . . . 8
7.2 State Descriptions . . . 8
7.2.1 APPROACH_CAVE . . . 8
7.2.2 EXPLORE_CAVE . . . 8
7.2.3 MISSION_COMPLETED . . . 9
7.3 State Transitions . . . 9
7.4 Communication Interfaces . . . 9
7.5 Design Rationale . . . 9
7.6 Robustness Considerations . . . 9

8 Results **10**
8.1 Mission Execution . . . 10
8.2 Lantern Detection Performance . . . 10

9 Current Limitations and Issues **10**

10 External Code Used **11**

11 Team Contributions **11**

1 Introduction

Briefly describe:

- Problem motivation
- Challenges of cave exploration
- Project objectives

2 System Architecture

The system is organized into modular ROS2 packages following a clear separation of concerns. The architecture consists of a high-level mission state machine, perception and mapping pipeline, trajectory generation module, lantern hunting module, and the provided quadrotor controller.

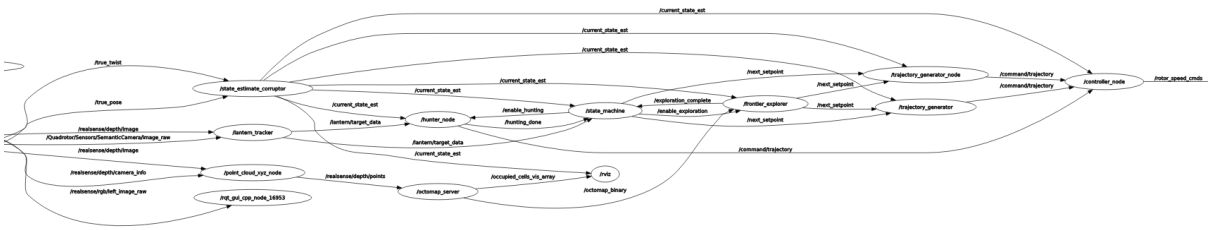

Figure 1: Nodes and Topics [Clear Version]

2.1 Implemented ROS Packages

We added the following custom packages:

- **decision_making** – High-level mission state machine
- **exploration_pkg** – Frontier-based autonomous cave exploration and backtracking logic.
- **trajectory_generator** – Polynomial trajectory generation
- **lantern_hunting** – Object interception logic
- **perception_pkg** – Depth to point cloud conversion, Octomap mapping, and lantern detection/logging.
- **teleop_pkg** – Manual control for testing and debugging.

The provided quadrotor controller was used without modification.

3 Trajectory Generation (trajectory_generator_pkg)

The trajectory generator node acts as the bridge between high-level navigation goals and the provided low-level quadrotor controller. It computes smooth, physically feasible flight paths by leveraging the open-source ETH Zürich MAV trajectory generation framework.

3.1 Method

Rather than manually computing flight paths, the node operates dynamically based on incoming data:

- **Input Processing:** The node subscribes to `/next_setpoint` (PoseStamped) to receive target goals from the state machine or frontier explorer. It also continuously tracks the drone's actual state via `/current_state_est` (Odometry).
- **Trajectory Optimization:** When a new goal is received, the node defines a start vertex (current position and velocity) and an end vertex (target position with zero terminal velocity). These are passed into the MAV library, which performs a 10th-order linear polynomial optimization constrained by predefined maximum velocity (10.0 m/s) and acceleration (2.0 m/s^2) limits.
- **Sampling:** The resulting continuous trajectory is sampled into a discrete set of trajectory points at a resolution of 100 Hz ($\Delta t = 0.01$ seconds).

3.2 Publishing

A timer-driven playback loop steps through the sampled trajectory points and sequentially publishes them at 100 Hz (every 10ms) to:

- `/command/trajectory` (MultiDOFJointTrajectory)

The provided quadrotor controller then consumes this topic to physically fly the drone along the generated path.

4 Perception and Mapping (perception)

The perception pipeline consists of two major components:

4.1 Depth to Point Cloud

The depth image is converted to a 3D point cloud using:

- `depth_image_proc`

4.2 Voxel Grid Mapping

We use Octomap to build a global voxel-grid representation of the cave.

- Resolution: 0.6
- Global map in world frame

The Octomap is reset before entering the cave to remove outside artifacts.
The resulting voxel grid is published on:

- `/octomap_full`

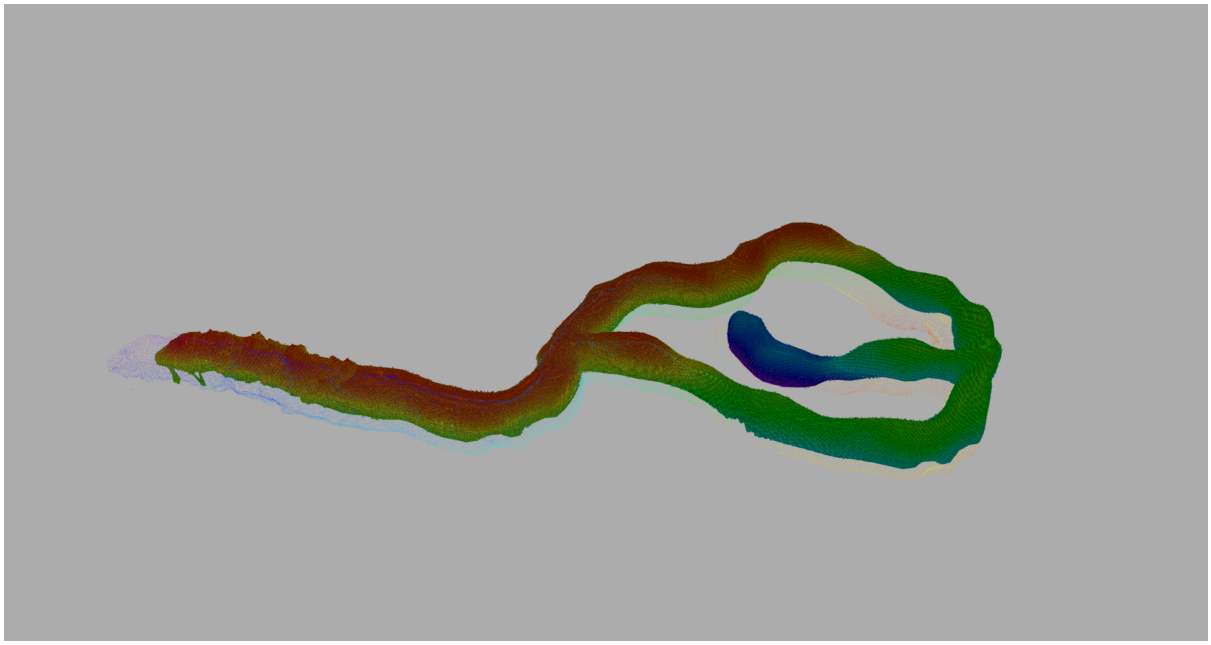

Figure 2: Generated Map

5 Lantern Detection and Hunting

Lantern detection is based on semantic camera data.

5.1 Detection

The perception module publishes:

- `/lantern/target_data` (Vector3)

  The y-component encodes detection confidence (area).
  A threshold of 200 is used to trigger hunting.

5.2 Hunting Strategy

Upon detection:

1. Exploration is disabled
2. Drone stabilizes for 2 seconds
3. Hunting node is enabled
4. Precision trajectory is generated

Once interception is complete, the hunting node publishes:

- `/hunting_done` (Bool)

The mission resumes exploration until four lanterns are secured.
Lantern positions are logged using current odometry.

6 Exploration Strategy (exploration_pkg)

The `frontier_explorer` node is the autonomous core of the system. It implements a **Topological Graph-based Frontier Exploration** strategy and maintains a spatial memory to guarantee collision-free backtracking even in complex cave networks.

6.1 Node Architecture

6.1.1 Frontier Detection

The drone searches the Octomap for *frontier voxels*.

- **Definition:** A voxel is a frontier if it is *Free* and adjacent to at least one *Unknown* voxel.
- **Optimization:** The search is restricted to a local bounding box

$$LOCAL_BOX_SIZE = 60.0\,m$$

  to ensure real-time performance.

6.1.2 Spatial Clustering

To generate navigation targets, frontier voxels are grouped using a Nearest-Neighbor Centroid Clustering algorithm:

1. Voxels are sorted by X-coordinate.
2. Voxels within

$$CLUSTER_DIST_THRESHOLD = 1.2\,m$$

   are grouped together.
3. For each cluster, the centroid is computed as the 3D mean.

Only clusters exceeding a minimum size are considered valid targets.

6.2 Advanced Exploration Strategy

6.2.1 Dynamic Threshold Reduction

Exploration starts with:

$$MIN_CLUSTER_SIZE = 150$$

If no valid clusters are found, the threshold is reduced iteratively by 30 until reaching `ABSOLUTE_MIN_SIZE`.

This allows prioritization of wide tunnels while still enabling exploration of narrow passages.

6.2.2 Heading Bias and Scoring

To reduce oscillatory behavior, candidate clusters are scored using directional alignment.

Let $\mathbf{v}_p$ be the drone's progress vector and $\mathbf{v}_c$ the vector toward a cluster. If:

$$\mathbf{v}_p \cdot \mathbf{v}_c > 0$$

the cluster score is multiplied by `HEADING_BIAS`.

This encourages forward motion deeper into the cave.

6.3 Navigation and Safety

6.3.1 Forbidden Radius Guard

A forbidden radius is defined as:

$$FORBIDDEN_RADIUS = 8.0\,m$$

Frontiers inside this radius of any existing graph node are ignored. This prevents local oscillations and repeated inspection of mapped areas.

6.3.2 Passive Logging Mode

When exploration is disabled by the state machine, the node enters *Passive Mode*.

- Every 5 meters, a raycast check (`isPathClear`) is performed toward the last graph node.
- If line-of-sight is lost (e.g., cornering), a Safety Node is inserted at the last valid position.

This guarantees that the backtracking chain remains intact even if control is temporarily handed over.

6.4 Graph Logic and Backtracking

The navigation graph is a tree structure. Each node stores:

- 3D position
- Parent ID
- A node is created whenever a frontier is reached.
- If no frontiers remain, the system switches to `BACKTRACKING`.
- Nodes are followed via parent links toward the entrance.
- If new frontiers are detected during return, the system resumes exploration.

6.5 Communication Interface

Subscribed Topics

- `/current_state_est` (`nav_msgs/msg/Odometry`)
- `/octomap_binary` (`octomap_msgs/msg/Octomap`)
- `/enable_exploration` (`std_msgs/msg/Bool`)

Published Topics

- `/next_setpoint` (`geometry_msgs/msg/PoseStamped`)
- `/exploration_complete` (`std_msgs/msg/Bool`)

7 Mission State Machine (decision_making)

The high-level mission coordination is implemented in the `state_machine_node`. The node operates as a timer-driven finite state machine (FSM) running at 2 Hz (500 ms period). Its responsibility is to coordinate navigation, exploration, and mission termination while interfacing with mapping and perception modules. The state machine ensures deterministic execution of the mission sequence and prevents conflicting behaviors between exploration,hunting, and navigation.

7.1 State Definition

The mission is modeled as a discrete finite state machine: Approach_Cave(s1),Explore_Cave(s2), Mission_Completed(s3).

$$S = \{s_1, s_2, s_3\}$$

The transition function $\delta : S \to S$ defines the evolution of the system.

7.2 State Descriptions

7.2.1 APPROACH_CAVE

This state consists of three sequential sub-steps:

1. Vertical climb to a predefined hover position
2. Horizontal navigation toward the cave entrance
3. Controlled entry into the cave interior

Waypoint transitions are triggered when the Euclidean distance between the current position $\mathbf{p}$ and the target position $\mathbf{p}_t$ satisfies:

$$\|\mathbf{p} - \mathbf{p}_t\| < \epsilon$$

where $\epsilon = 1.5$ m.

Before entering the cave, the Octomap is reset using the `/octomap_server/reset` service. This ensures that only the interior cave structure is mapped and prevents outside artifacts from influencing frontier exploration.

Exploration is explicitly disabled during this phase.

7.2.2 EXPLORE_CAVE

After entering the cave, a 4-second waiting period is enforced to allow depth sensors and the Octomap to accumulate stable environment data following the reset.

Once the warm-up period has elapsed, the state machine publishes:

- `/enable_exploration` = True

Exploration continues until a completion signal is received:

- `/exploration_complete` (Bool)

Upon receiving a positive completion message, the state machine transitions to the final mission state.

7.2.3 MISSION_COMPLETED

In the terminal state, exploration is disabled and no further goals are issued. The drone remains in hover at its current location.

7.3 State Transitions

The transition function can be summarized as:

$$\delta(s_1) = \begin{cases} s_2 & \text{if cave entry reached} \\ s_1 & \text{otherwise} \end{cases}$$

$$\delta(s_2) = \begin{cases} s_3 & \text{if exploration_complete} = \text{True} \\ s_2 & \text{otherwise} \end{cases}$$

The state transitions are therefore deterministic and triggered either by geometric proximity conditions or Boolean completion signals.

7.4 Communication Interfaces

The state machine interacts with other modules via the following ROS2 interfaces:

Published Topics

- `/next_setpoint` (PoseStamped)-Publishes the next desired goal pose for trajectory generation and control.
- `/enable_exploration` (Bool)-Enables or disables the frontier exploration module.

Subscribed Topics

- `/current_state_est` (Odometry)
- `/exploration_complete` (Bool)

Service Clients

- `/octomap_server/reset`

7.5 Design Rationale

A timer-driven architecture was chosen to ensure periodic and deterministic evaluation of state transitions. This simplifies implementation and provides predictable behavior even under delayed sensor updates. The waypoint tolerance of 1.5 m provides a balance between precision and stability, avoiding oscillations near goal positions. Resetting the Octomap before cave entry isolates the interior environment and prevents external structures from influencing exploration.A short waiting period after reset allows sensor data to stabilize before enabling frontier exploration.

7.6 Robustness Considerations

Several mechanisms improve system robustness:

- Odometry availability is verified before state evaluation.
- The Octomap reset service is called only if available.
- Waypoints are transmitted only once per sub-step to prevent message flooding.

- Exploration is explicitly disabled during navigation phases to avoid conflicting goal commands.

These measures ensure stable and predictable mission execution.

8 Results

8.1 Mission Execution

The system successfully completed the following mission stages fully autonomously:

- Autonomous take-off and stabilization
- Navigation to the cave entrance
- Reset and initialization of the Octomap mapping
- Autonomous exploration inside the cave
- Detection and localization of three out of four lanterns
- Safe flight throughout the mission without crashes

The voxel-grid representation of the cave was successfully generated using Octomap and continuously updated during exploration.

8.2 Lantern Detection Performance

The system successfully detected and secured three lanterns. Each detection triggered a stabilization phase followed by a dedicated hunting maneuver.

The fourth lantern was not detected due to incomplete exploration of a specific branch of the cave (see Limitations section).

All detections and interception maneuvers were performed autonomously.

9 Current Limitations and Issues

Although the system operated fully autonomously and safely, several limitations were observed:

- The exploration strategy is fully able to explore the cave without implementing the Lantern Hunting mode in current version of the code.
- To deal with this issue multiple exploration strategies were tested including ERRT(Exploration-RRT),NBV (Next Best View), but we were unable to integrate it properly with the lantern detection part.
- Importantly, lantern detection itself is robust and not sensitive to lighting conditions. The missed lantern was solely due to incomplete spatial exploration.

Future improvements could focus on more systematic frontier-based coverage planning to guarantee full exploration of branching structures.

10 External Code Used

The following external libraries were used:

- MAV Trajectory Generation (ETH Zürich)
- Octomap (3D occupancy grid mapping)
- depth_image_proc (Depth image to point cloud conversion)

All high-level mission logic, state machine implementation, exploration coordination, and integration were implemented by the team.

11 Team Contributions

— **Fuzail Hamid Banday** – Decision Making Package, Git Management + (lots of unsuccessful tries of exploration strategies)

— **Firdavs Shukurilloev** – Teleop Package, Exploration Package (Frontier-based exploration with RRT*)

— **Samuele Ribaudo** – Trajectory Generation Package, Exploration Package (Graph-based exploration attempts)

— **Hong Yan Jun** – Perception Package, Lantern Detection, Decision Making Package

— **Om Anant Kulkarni** – Decision Making Package, Exploration Package,(ERRT,RTAB-Map implementation attempts)